VIVA GUIDE 05 / 05

Architecture, operations, and verification

How the services fit together, how to run a demo, and what the tests and limitations mean.

YOUR RESPONSIBILITY

Own the integrated system view: service boundaries, local operation, testing strategy, and honest scope.

THE SHORT VERSION

The project is a three-service application: React/Vite presents the interface, Express owns browser-facing APIs and sessions, and FastAPI retrieves NIT passages and calls Gemini. The system is deliberately a local academic implementation; its boundaries and limitations should be explained accurately.

A 30-SECOND OPENING

I explain how the pieces work as one system. The browser does not call Gemini directly; Express manages sessions and relays the question; FastAPI retrieves official passages and requests generation. Each boundary has a health signal and a clear failure mode.

IN THIS GUIDE

- End-to-end architecture
- Local setup and operating sequence
- Verification and demo plan
- Why these design choices fit the project
- Security, reliability, and scope boundaries
- Five-minute presentation outline
- Likely viva questions

END-TO-END ARCHITECTURE

```
Student
| React + Vite
| Axios: /api/chat
v
Express API :5000
| session JSON store
| Axios: /api/v1/nlp/chat
v
FastAPI :8000
| local NIT passage index
| HTTPS provider request
v
Gemini generateContent

Answer + cited passages return along the same path.
```

The separation keeps responsibilities understandable: presentation and interaction in the client; session persistence and orchestration in Express; retrieval, prompt construction, and model communication in FastAPI. The document indexing script is a separate build-time/offline path that produces the index consumed by FastAPI.

- Frontend: React/Vite, Axios, ReactMarkdown, lucide-react.
- Backend: Express, local JSON persistence, Axios, rate limiting, Multer upload handling.
- AI service: FastAPI/Pydantic, local page-aware index, lexical retriever, Gemini REST request.
- Document prep: Python script with PyMuPDF and an official-source allowlist.

LOCAL SETUP AND OPERATING SEQUENCE

Use the repository README and each service's package/requirements files as the source of truth for install commands. A normal local run has three processes: frontend development server, Express API, and FastAPI AI service. The browser talks to Express; Express needs the AI service URL; FastAPI needs its document index and, for matched questions, the Gemini key.

- Prepare the Python AI environment and install its declared requirements.
- Configure GEMINI_API_KEY server-side in ai-service/.env; never put the key in client/.env or source control.
- Run the official NIT ingestion script when the local nith_documents.json index is absent or needs refreshing; network access to official PDFs is required.
- Start FastAPI on port 8000 and confirm /api/v1/health reports index readiness.
- Start Express on port 5000 and confirm /api/health reports Express, AI, Gemini configuration, and index status.
- Start the Vite client and test a question covered by a configured document.

Health is layered: Express can be running while FastAPI is down; FastAPI can be running while Gemini is missing; a non-empty Gemini key can still be invalid.

VERIFICATION AND DEMO PLAN

The repository has backend tests, AI-service tests, and client lint/build scripts. The AI tests use mocked provider behavior where appropriate, so passing them verifies application logic without proving a real Gemini account/key/network combination works. No live Gemini key was supplied for this workspace, so an authenticated provider request has not been verified.

- Run the backend test script from server and the AI test suite from ai-service; use the client lint and production-build scripts from client.
- Use health endpoints to verify service wiring before testing chat.
- Test a relevant question and inspect the page-aware source list.
- Test an unrelated question and confirm the grounded not-found response.
- Test missing key/provider failure separately and explain the difference between HTTP health and end-to-end generation.
- For a final viva demo, use a known indexed fact and open its original official source.

A useful test matrix separates components: session route behavior, chat validation and fallback, retrieval ranking/no-match behavior, Gemini request formatting and error mapping, and frontend production compilation. A mocked Gemini response proves code paths and response parsing, not provider availability.

WHY THESE DESIGN CHOICES FIT THE PROJECT

- Separate processes allow the UI and API to evolve independently and make AI-specific errors easier to isolate.
- Local JSON persistence reduces setup friction for a student demo and makes stored sessions inspectable.
- An offline index avoids downloading official PDFs for every question.
- Lexical retrieval and a REST provider call keep the prototype lightweight and straightforward to explain.
- Explicit source metadata gives students a way to verify generated answers.

These are prototype trade-offs, not universal production recommendations. As usage grows, the file store could move to a transactional database; retrieval could use embeddings and a vector store; indexing could become a durable background job; access control and monitoring could be added.

SECURITY, RELIABILITY, AND SCOPE BOUNDARIES

- Keep the Gemini key in server-side environment configuration; rotate it if it is exposed.
- The Express admin upload routes have no visible authentication in this build and should not be publicly exposed as-is.
- JSON file read-modify-write persistence is not designed for multi-process concurrency or production durability.
- The UI and API have timeouts, but a provider outage still prevents generated answers.
- Uploaded document metadata is not the same as active RAG indexing; explain the current stub accurately.
- Feedback currently validates and echoes input but does not store a durable review record.
- The retrieval corpus is a small, selected document set, not the complete or continuously updated institute knowledge base.

A 502 on the AI route is a generation-path error, not evidence that the whole application failed to start. Check the AI service logs and response detail, key/model configuration, network access, and whether the question actually retrieved evidence. A health endpoint showing a configured key only checks that the environment variable is non-empty.

FIVE-MINUTE PRESENTATION OUTLINE

- 1. State the problem: students need a simple way to locate information scattered across official institute PDFs.
- 2. Show the architecture diagram and name the responsibility of each service.
- 3. Ask a document-covered question; point out that the UI shows the source title and page.
- 4. Open the source PDF and verify the page, explaining that citations are a verification path.
- 5. Mention a no-match question and explain that the service does not call Gemini without retrieved evidence.

- 6. Finish with one limitation and one next step: lexical retrieval today; semantic retrieval and persistent production storage are possible future improvements.

Do not claim MongoDB, Chroma/vector search, OCR, dynamic PDF uploads into the live index, or verified live Gemini access unless those capabilities have actually been implemented and tested.

LIKELY VIVA QUESTIONS

What problem does this application solve?
It helps students ask questions against a selected set of official NIT Hamirpur documents and inspect the evidence used.

Why are there three services?
They isolate browser experience, session/API orchestration, and document retrieval/model generation into clear responsibilities.

What is the source of truth for an answer?
The retrieved official document excerpts; the model is instructed not to invent information beyond them.

Is the system production-ready?
No. It is a working academic/local implementation with JSON persistence, a small snapshot corpus, lexical retrieval, and no shown admin authentication.

What does a passing test suite prove?
It proves covered code paths behave under the test conditions; provider mocks do not validate a real key or external service.

What would you improve first?
Add authenticated admin workflows and durable storage, then improve retrieval evaluation and semantic matching while retaining citations.

Can the app answer follow-up questions using chat history?
The conversation is stored and sent by Express, but FastAPI currently uses only the current question, so contextual follow-up reasoning is not implemented.